Understanding Blockchain Through ProofPay

A hands-on tour of core blockchain concepts, taught through the actual smart contract, backend, and frontend in this repository's milestone-escrow app.

This guide uses **ProofPay** — the blockchain milestone-escrow app for freelancers in this repository — as a running example. Every concept below is tied to a real file: `contracts/contracts/ProofPayEscrow.sol`, the NestJS backend, and the Next.js frontend. The goal is not abstract theory, but to let you point at a line of working code and say “that's what a smart contract is” or “that's what an event is for.”

What ProofPay does, in one paragraph:

A client and a freelancer agree on work broken into milestones. The client deposits funds into a smart contract — not into ProofPay's company bank account, not into a database row, but into code running on a blockchain that nobody (including ProofPay's own backend) can unilaterally touch. As the freelancer completes each milestone, funds move step by step: submit → approve → release. If the client and freelancer disagree, a neutral arbitrator steps in. Every one of these actions is a transaction signed by someone's own wallet.

1. The Core Idea: A Ledger Nobody Owns Alone

The single idea underneath every blockchain buzzword is this: instead of one company's database being the trusted record of “who owns what,” the record lives on a network of computers that all run the same rules and agree on the same history. No single party can quietly edit a past entry, and no single party can move funds without following the rules everyone else is also enforcing.

ProofPay makes this concrete in one sentence from its own README:

> **From README.md**
>
> “The backend never moves funds. Every state-changing action (create escrow, deposit, add/submit/approve a milestone, release, dispute, refund) is a transaction signed by the user's own wallet in the frontend.”

That is a blunt, practical version of “trustless.” ProofPay's own servers are deliberately not trusted with money. If you wanted to be evil and run away with escrowed funds, hacking the NestJS backend wouldn't get you anywhere — the funds live in the smart contract, which only obeys its own code, not the backend's requests.

The three layers

```
Next.js Frontend (wagmi/viem, wallet-signed txs)
        |
        v
NestJS API (Postgres + Redis)  <-- read-only chain access, never holds a signer
        ^
        | events
ProofPayEscrow.sol (Hardhat/Solidity)  <-- sole custodian of escrowed funds
```

Only the bottom layer, the smart contract, actually holds money. The other two layers exist to make the on-chain data pleasant to use: fast search, notifications, a proper multi-page UI. Section 8 explains how the backend keeps its copy of the truth in sync.

2. Wallets, Addresses, and Signing

A blockchain doesn't have “user accounts” with passwords. It has **addresses** — derived from a private key that only the owner holds — and every action taken “as” an address must be **signed** with that private key. ProofPay's README instructs you to import a Hardhat test account's private key into MetaMask to act as the client, and a second one to act as the freelancer (and a third for the arbitrator, needed now that the app has a full dispute-resolution flow).

- **Private key** — a secret number only the owner should ever hold. Whoever has it can sign transactions as that address. MetaMask hides this behind a browser extension.
- **Address** — a public identifier derived from the key (e.g. 0xabc...). In the contract, `escrows[escrowId].client` and `.freelancer` are just addresses.
- **Signing a transaction** — proving “I authorize this specific action” without ever revealing the private key itself. When the frontend calls `deposit()` via wagmi/viem, MetaMask pops up asking the user to sign — that signature is what the network checks.

This is why the contract can trust `msg.sender` completely: it is not a cookie or a JWT that could be forged, it is the address that cryptographically signed the incoming transaction. That single fact is what makes access control like `onlyClient` meaningful.

> **A hazard worth knowing about**
>
> Wallets don't automatically know which network you mean an address for — they'll happily sign a transaction on whichever chain is currently selected. Sending a test-only call while your wallet is still pointed at Ethereum mainnet is a real (and free to make) mistake; ProofPay's frontend now pins every write to chain 31337 explicitly so the wallet is forced to switch (or refuse) rather than sign silently on the wrong network.

3. Smart Contracts: Code as the Rulebook

A smart contract is a program deployed to the blockchain whose code becomes the permanent, public rulebook for some piece of state. `ProofPayEscrow.sol` is ProofPay's rulebook for one thing: who can move escrowed money, and under what conditions.

Two structs define the entire data model:

```
struct Escrow {
    address client;
    address freelancer;
    address arbitrator;
    uint256 balance; // deposited funds not yet released or refunded
}

struct Milestone {
    uint256 amount;
    MilestoneStatus status;
    bytes32 proofHash;
}
```

Notice what is *not* here: no company, no admin key, no “ProofPay” address with special powers. The only privileged parties are the three addresses baked into the escrow itself at creation time — client, freelancer, and arbitrator — enforced entirely by code, not by a company's internal policy.

Once `ProofPayEscrow.sol` is deployed (via `npm run deploy:local`), its logic is fixed. Nobody — not ProofPay, not the client, not the freelancer — can quietly change what `releasePayment` does. That immutability is the whole point: it's what lets a client trust a freelancer they've never met.

4. The Milestone Lifecycle: A State Machine

Each milestone moves through a strict sequence of statuses. A function can only move a milestone forward if it is currently in the exact status that function expects — this is what stops a freelancer from “approving” their own work, or a client from releasing funds before work is submitted.

Status	Set by function	Who can call it	Meaning
Pending	addMilestone	Client	Milestone exists, no work submitted yet
Submitted	submitMilestone	Freelancer	Work delivered; proofHash recorded on-chain
Approved	approveMilestone	Client	Client accepts the work as complete
Released	releasePayment	Anyone (once Approved)	Funds paid to the freelancer
Disputed	openDispute	Client or freelancer	Either party contests before release
Refunded	refund	Arbitrator only	Disputed funds returned to the client

The contract enforces this with plain `require()` checks — for example, `approveMilestone` refuses to run unless the milestone's status is exactly `Submitted`:

```
function approveMilestone(uint256 escrowId, uint256 milestoneId)
    external escrowExists(escrowId) onlyClient(escrowId)
{
    Milestone storage milestone = _milestone(escrowId, milestoneId);
    require(milestone.status == MilestoneStatus.Submitted, "milestone not submitted");

    milestone.status = MilestoneStatus.Approved;
    emit MilestoneApproved(escrowId, milestoneId);
}
```

This is the general pattern behind almost every blockchain application: define a finite set of states, and write functions that only allow specific, checked transitions between them.

5. Where the Money Actually Lives

When a client calls `deposit()`, they send real value (ETH, or a test chain's native currency) directly to the contract's own address, not to any bank account:

```
function deposit(uint256 escrowId) external payable escrowExists(escrowId) onlyClient(
escrowId) {
    require(msg.value > 0, "deposit must be > 0");
    escrows[escrowId].balance += msg.value;
    emit Deposited(escrowId, msg.sender, msg.value);
}
```

The `payable` keyword is what allows a function to receive native currency; `msg.value` is the amount attached to the transaction. The contract itself now custodies that balance in its own storage — `escrows[escrowId].balance` — until a later call sends it onward.

Releasing payment does the reverse: it pushes value out to the freelancer's address using a low-level call, then checks that it actually succeeded:

```
(bool success, ) = payable(escrow.freelancer).call{value: amount}("");
require(success, "transfer to freelancer failed");
```

Guarding against reentrancy

Notice that `releasePayment` decrements `escrow.balance` **before** sending the funds out, and the whole function is wrapped in a `nonReentrant` modifier. This defends against a classic smart-contract exploit: if the freelancer's address were a malicious contract, its fallback code could try to call back into `releasePayment` again *during* the transfer, draining the escrow before its balance was ever updated. Updating state first, and locking the function while it runs, closes that door.

```
modifier nonReentrant() {
    require(!locked, "reentrant call");
    locked = true;
    _;
    locked = false;
}
```

6. Access Control: Who Is Allowed to Do What

Every sensitive function is gated by a **modifier** that checks `msg.sender` against an address stored in the escrow. This is the on-chain equivalent of a permissions system, except it cannot be bypassed by attacking a server — there is no server in the loop for these checks.

```
modifier onlyClient(uint256 escrowId) {
    require(msg.sender == escrows[escrowId].client, "not client");
    _;
}
```

- **Client** — creates the escrow, deposits funds, adds milestones, approves submitted work.
- **Freelancer** — submits milestones with a proof hash, receives released payments.
- **Arbitrator** — a neutral third address, chosen when the escrow is created, who alone can resolve a dispute by releasing funds to the freelancer or refunding the client.

This is worth pausing on: the README notes that the backend currently does not re-check that a signed-in wallet matches the on-chain client/freelancer for a given action — and explains that this is safe only because “the smart contract already enforces this on-chain for anything that moves money.” That's the trust model in a nutshell: the chain is the actual security boundary; the backend (and the frontend, which now hides buttons a connected wallet isn't allowed to use) is only a convenience layer on top of it.

7. Events: The Contract's Public Diary

Every meaningful action in `ProofPayEscrow.sol` emits an **event** — a permanently recorded log entry attached to the transaction that triggered it:

```
event EscrowCreated(uint256 indexed escrowId, address indexed client, address indexed
freelancer, address arbitrator);
event Deposited(uint256 indexed escrowId, address indexed from, uint256 amount);
event MilestoneSubmitted(uint256 indexed escrowId, uint256 indexed milestoneId, bytes3
2 proofHash);
event PaymentReleased(uint256 indexed escrowId, uint256 indexed milestoneId, address i
ndexed to, uint256 amount);
event DisputeOpened(uint256 indexed escrowId, uint256 indexed milestoneId, address ind
exed openedBy);
event Refunded(uint256 indexed escrowId, uint256 indexed milestoneId, address indexed
to, uint256 amount);
```

Events aren't just logging for humans — they're the interface the rest of ProofPay's stack uses to know what happened on-chain, without ever trusting a client's word for it. The frontend's Events tab on each escrow page shows this raw log directly, exactly as the indexer captured it — see Section 8.

8. Keeping an Off-Chain Copy in Sync

A blockchain is a poor fit for the kind of fast, flexible queries a web app needs ("show me all escrows for this user, sorted by date"). ProofPay's Postgres database exists purely as a fast, queryable *mirror* of on-chain truth — never the other way around. It gets kept in sync two ways:

- **Immediate sync** — right after a transaction confirms, the frontend posts just the `txHash` to the backend. The backend fetches the real receipt from the chain, decodes the event from it, and writes the row. It never trusts amounts or statuses sent in the request body — only what it reads back from the chain itself. Every state-changing action has this endpoint now, including release and refund, which originally only relied on the background indexer.
- **Background sync** — `IndexerService` (`backend/src/indexer`) subscribes to the contract's events continuously and replays anything it missed since the last block it saw (tracked in Redis), so the database self-heals even if a client's confirmation request never arrives.

This pattern — *on-chain data is the source of truth; a database is only a cache of it* — is one of the most common architectures in real blockchain applications, and ProofPay implements it directly. The frontend's Activity tab (backed by a new `GET /escrows/:onChainId/audit-logs` endpoint) makes this visible: it shows every off-chain action taken against a given escrow, timestamped, separate from the raw on-chain Events tab.

9. Disputes: What Happens When Trust Breaks Down

Escrow only matters when things might go wrong. If a client refuses to approve legitimate work, or a freelancer submits something unfinished, either party can call `openDispute`, moving the milestone into the `Disputed` state. From there, only the arbitrator — an address both sides already agreed on when the escrow was created — can resolve it, by calling either `releasePayment` (pays the freelancer) or `refund` (returns funds to the client).

Notice that even dispute resolution goes through the same two functions as the happy path — the contract just relaxes the required status check when `msg.sender` is the arbitrator. There's no separate, less-audited "admin panel" code path; it's the same rulebook.

> **Now visible end to end in the app**
>
> The frontend has a dedicated Disputes tab per escrow (and a cross-escrow Disputes page in the sidebar): either party can open a dispute, both sides can submit evidence (a note plus an optional content hash — the same hash-only pattern used for proof-of-delivery), and the arbitrator resolves it from the Milestones tab with a single click, which signs the exact `releasePayment`/`refund` call shown above.

10. Local Chains, Testnets, and Mainnet

ProofPay is designed to run entirely against a local blockchain: `npm run node` starts a private Hardhat chain on `127.0.0.1:8545` with chain ID `31337`, pre-funded with test accounts whose private keys are public and meant only for development. Deploying is just another transaction — `npm run deploy:local` writes the resulting contract address and ABI to `deployments/localhost.json`, which the backend and frontend then point at.

Local chain vs. a public network

> A local Hardhat node behaves like a real blockchain (blocks, transactions, gas) but only your machine can see it, and resetting it wipes all history. A public testnet (e.g. Sepolia) is shared and persistent but still uses worthless test currency. Mainnet is the real, permanent, publicly shared ledger where transactions cost real money and cannot be undone — which is exactly why ProofPay's own README says: **local/testnet only, do not point this at mainnet or handle real funds.**

11. Walking the Full Flow End to End

The frontend used to be a single scrolling page; it's now a proper multi-page app — a Dashboard listing every escrow your wallet is party to, a New Escrow form, and a per-escrow page with Milestones / Disputes / Transactions / Events / Activity tabs — but the underlying sequence of contract calls hasn't changed at all. Here is that same sequence read purely from the contract's function names:

```
1. createEscrow(freelancer, arbitrator)      -- client picks the two other parties
2. deposit(escrowId)                         -- client sends funds into the contract
3. addMilestone(escrowId, amount)            -- client defines a unit of work + its pr
ice
4. submitMilestone(escrowId, id, hash)       -- freelancer delivers, hash = proof of t
he work
5. approveMilestone(escrowId, id)            -- client signs off
6. releasePayment(escrowId, id)              -- funds move to the freelancer, on-chain
   (or, if disputed:)
   openDispute(escrowId, id) -> arbitrator calls releasePayment() or refund()
```

Every single one of those six-plus calls is a wallet-signed transaction, mined into a block, emitting an event, permanently recorded. Nothing about “who paid whom” ever depends on ProofPay's own servers staying honest or even staying online — and every one of those steps now has a matching screen in the app: the Milestones tab drives steps 1–6, and the Disputes tab drives the dispute branch, evidence and all.

12. Glossary (Mapped to ProofPay)

Term	What it means here
Wallet	MetaMask (or similar) — holds a private key and signs transactions on the user's behalf.
Address	A public identifier like the client/freelancer/arbitrator fields on an Escrow.
Transaction (tx)	A signed, on-chain call such as deposit() or approveMilestone(); each has a txHash.
Smart contract	ProofPayEscrow.sol — deployed code that is the sole custodian of escrowed funds.
Gas	The fee paid to the network to execute a transaction; free/test ETH on the local Hardhat chain.
Event	An on-chain log (e.g. PaymentReleased) that off-chain systems use as their source of truth.
Custody	Who actually holds the funds — here, the contract itself, never ProofPay's backend.
Indexer	IndexerService — replays contract events into Postgres so the app can query state fast.
Reentrancy	An exploit where an external call re-enters a function mid-execution; blocked by nonReentrant.
Audit log	Off-chain record of who took what action and when, viewable per-escrow in the Activity tab.
Testnet / local chain	A throwaway blockchain (here, Hardhat on chain ID 31337) using worthless test funds.
Mainnet	The real, permanent public network — explicitly out of scope for this project today.

Further reading in this repo: README.md (architecture, setup, and the backend API additions made beyond the original spec), ProofPay_Blockchain_Escrow_Project_Specification.pdf (full product spec), and contracts/contracts/ProofPayEscrow.sol (the rulebook itself — it's under 210 lines and worth reading top to bottom now that you know what each piece does).